ACTIVITY SHEET – 10

Threads, Applets, Event Handling & GUI Components

1. OBJECTIVES

• Understand thread creation and lifecycle
• Apply multithreading concepts
• Implement event-driven programming
• Work with GUI components and event handling

2. PROGRAMMING / TASK QUESTIONS

A. EASY LEVEL

1. A program creates two threads: one prints numbers 1–5 and another prints 100–105. Explain why output order varies and modify to execute sequentially.
Hint: Use join().

JAVA CODE:

Welcome | ThreadSequential.java

ThreadSequential.java > ...

```java
import java.util.Scanner;
class NumberThread extends Thread {
    int start, end;
    NumberThread(int s, int e) {
        start = s;
        end = e;
    }
    public void run() {
        for (int i = start; i <= end; i++) {
            System.out.print(i + " ");
            try {
                Thread.sleep(millis: 100);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
        System.out.println();
    }
}
public class ThreadSequential {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print(s: "Enter start and end for Thread 1: ");
        int s1 = sc.nextInt();
        int e1 = sc.nextInt();
        System.out.print(s: "Enter start and end for Thread 2: ");
        int s2 = sc.nextInt();
        int e2 = sc.nextInt();
        sc.close();
```

```
Welcome        ThreadSequential.java ●
ThreadSequential.java > ...
public class ThreadSequential {
    public static void main(String[] args) {
        NumberThread t1 = new NumberThread(s1, e1);
        NumberThread t2 = new NumberThread(s2, e2);
        t1.start();
        try {
            t1.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        t2.start();
        try {
            t2.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        System.out.println(x: "Execution Completed Sequentially");
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>javac ThreadSequential.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>java ThreadSequential.java
Enter start and end for Thread 1: 1 5
Enter start and end for Thread 2: 100 105
1 2 3 4 5
100 101 102 103 104 105
Execution Completed Sequentially
```

2. Design a GUI with a text field and button. If field is empty show error, else display input.
Hint: Use event handling and conditions.

JAVA CODE:

Welcome | ThreadSequential.java | TextFieldValidation.java

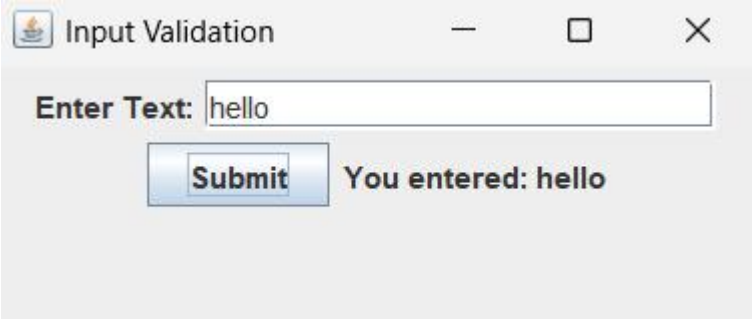
TextFieldValidation.java > ...

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class TextFieldValidation {
    Run | Debug
    public static void main(String[] args) {
        JFrame frame = new JFrame(title: "Input Validation");
        frame.setSize(width: 400, height: 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new FlowLayout());
        JTextField textField = new JTextField(columns: 20);
        JButton button = new JButton(text: "Submit");
        JLabel resultLabel = new JLabel(text: "");
        frame.add(new JLabel(text: "Enter Text:"));
        frame.add(textField);
        frame.add(button);
        frame.add(resultLabel);
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String input = textField.getText();
                if (input.trim().isEmpty()) {
                    resultLabel.setText(text: "Error: Field cannot be empty!");
                } else {
                    resultLabel.setText("You entered: " + input);
                }
            }
        });
        frame.setVisible(b: true);
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>javac TextFieldValidation.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>java TextFieldValidation.java
```



3. Compare execution of a task using normal method and using threads.
Hint: Sequential vs concurrent execution.

JAVA CODE:

Welcome | ThreadSequential.java | TextFieldValidation.java | ThreadComparisonInput.java

ThreadComparisonInput.java > ThreadComparisonInput > main(String[])

```java
import java.util.Scanner;
class Task {
    void printRange(int start, int end, String name) {
        for (int i = start; i <= end; i++) {
            System.out.println(name + ": " + i);
            try {
                Thread.sleep(millis: 500);
            } catch (InterruptedException e) {}
        }
    }
}
class MyThread extends Thread {
    int start, end;
    String name;
    MyThread(int s, int e, String n) {
        start = s;
        end = e;
        name = n;
    }
    public void run() {
        for (int i = start; i <= end; i++) {
            System.out.println(name + ": " + i);
            try {
                Thread.sleep(millis: 500);
            } catch (InterruptedException e) {}
        }
    }
}
```

Welcome | ThreadSequential.java | TextFieldValidation.java | ThreadComparisonInput.java

ThreadComparisonInput.java > ...

```java
public class ThreadComparisonInput {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print(s: "Enter start and end for Task 1: ");
        int s1 = sc.nextInt();
        int e1 = sc.nextInt();
        System.out.print(s: "Enter start and end for Task 2: ");
        int s2 = sc.nextInt();
        int e2 = sc.nextInt();
        sc.close();
        Task obj = new Task();
        System.out.println(x: "\n=== Sequential Execution ===");
        long start1 = System.currentTimeMillis();
        obj.printRange(s1, e1, name: "Task1");
        obj.printRange(s2, e2, name: "Task2");
        long end1 = System.currentTimeMillis();
        System.out.println("Time Taken: " + (end1 - start1) + " ms");
        System.out.println(x: "\n=== Concurrent Execution (Threads) ===");
        long start2 = System.currentTimeMillis();
        MyThread t1 = new MyThread(s1, e1, n: "Thread1");
        MyThread t2 = new MyThread(s2, e2, n: "Thread2");
        t1.start();
        t2.start();
        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {}
        long end2 = System.currentTimeMillis();
        System.out.println("Time Taken: " + (end2 - start2) + " ms");
    }
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>javac ThreadComparisonInput.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>java ThreadComparisonInput.java
Enter start and end for Task 1: 1 5
Enter start and end for Task 2: 100 110

=== Sequential Execution ===
Task1: 1
Task1: 2
Task1: 3
Task1: 4
Task1: 5
Task2: 100
Task2: 101
Task2: 102
Task2: 103
Task2: 104
Task2: 105
Task2: 106
Task2: 107
Task2: 108
Task2: 109
Task2: 110
Time Taken: 8037 ms
```

```
=== Concurrent Execution (Threads) ===
Thread1: 1
Thread2: 100
Thread2: 101
Thread1: 2
Thread2: 102
Thread1: 3
Thread1: 4
Thread2: 103
Thread1: 5
Thread2: 104
Thread2: 105
Thread2: 106
Thread2: 107
Thread2: 108
Thread2: 109
Thread2: 110
Time Taken: 5537 ms
```

B. MEDIUM LEVEL

4. Simulate file download where one thread shows progress (0–100%) and another displays it.

Hint: Use sleep().

JAVA

CODE:

Welcome | ThreadSequential.java | TextFieldValidation.java | ThreadCompar

FileDownloadSimulation.java > ...

```java
class Download {
    int progress = 0; // shared variable
}
class Downloader extends Thread {
    Download d;
    Downloader(Download d) {
        this.d = d;
    }
    public void run() {
        for (int i = 0; i <= 100; i++) {
            d.progress = i;
            try {
                Thread.sleep(millis: 100); // simulate download time
            } catch (InterruptedException e) {}
        }
    }
}
class ProgressDisplay extends Thread {
    Download d;
    ProgressDisplay(Download d) {
        this.d = d;
    }
```

Welcome | ThreadSequential.java | TextFieldValidation.java | ThreadCompariso

FileDownloadSimulation.java > ...

```java
class ProgressDisplay extends Thread {

    public void run() {
        while (true) {
            System.out.println("Download Progress: " + d.progress + "%");
            if (d.progress >= 100) {
                break;
            }
            try {
                Thread.sleep(millis: 200); // display delay
            } catch (InterruptedException e) {}
        }
    }
}

public class FileDownloadSimulation {
    Run | Debug
    public static void main(String[] args) {
        Download d = new Download();
        Downloader t1 = new Downloader(d);
        ProgressDisplay t2 = new ProgressDisplay(d);
        t1.start();
        t2.start();
        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {}
        System.out.println(x: "Download Completed!");
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>javac FileDownloadSimulation.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>java FileDownloadSimulation.java
Download Progress: 0%
Download Progress: 2%
Download Progress: 4%
Download Progress: 6%
Download Progress: 7%
Download Progress: 9%
Download Progress: 12%
Download Progress: 15%
Download Progress: 17%
Download Progress: 19%
Download Progress: 21%
Download Progress: 25%
Download Progress: 27%
Download Progress: 29%
Download Progress: 31%
Download Progress: 33%
Download Progress: 35%
Download Progress: 37%
Download Progress: 39%
Download Progress: 41%
Download Progress: 43%
Download Progress: 45%
Download Progress: 47%
Download Progress: 49%
Download Progress: 51%
Download Progress: 53%
```

```
Download Progress: 45%
Download Progress: 47%
Download Progress: 49%
Download Progress: 51%
Download Progress: 53%
Download Progress: 55%
Download Progress: 57%
Download Progress: 59%
Download Progress: 61%
Download Progress: 63%
Download Progress: 66%
Download Progress: 68%
Download Progress: 70%
Download Progress: 72%
Download Progress: 75%
Download Progress: 77%
Download Progress: 79%
Download Progress: 81%
Download Progress: 84%
Download Progress: 86%
Download Progress: 88%
Download Progress: 90%
Download Progress: 92%
Download Progress: 95%
Download Progress: 97%
Download Progress: 99%
Download Progress: 100%
Download Completed!
```

5. Design a GUI calculator using JTextField, JComboBox and JButton.
Hint: Use ActionListener.

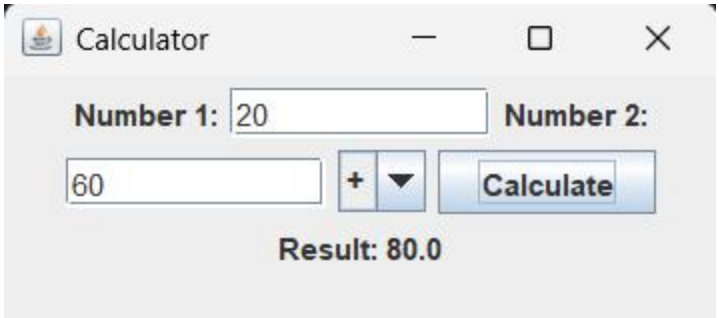
JAVA CODE:

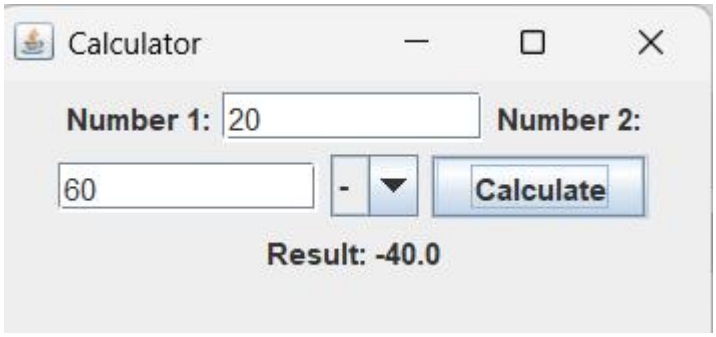
```
Welcome | ThreadSequential.java | TextFieldValidation.java | ThreadCompa
GUICalculator.java > GUICalculator > main(String[]) > new ActionListener() {...} > action
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class GUICalculator {
    Run | Debug
    public static void main(String[] args) {
        JFrame frame = new JFrame(title: "Calculator");
        frame.setSize(width: 300, height: 200);
        frame.setLayout(new FlowLayout());
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JTextField num1 = new JTextField(columns: 10);
        JTextField num2 = new JTextField(columns: 10);
        String operations[] = {"+", "-", "*", "/"};
        JComboBox<String> operator = new JComboBox<>(operations);
        JButton calculate = new JButton(text: "Calculate");
        JLabel result = new JLabel(text: "Result: ");
        frame.add(new JLabel(text: "Number 1:"));
        frame.add(num1);
        frame.add(new JLabel(text: "Number 2:"));
        frame.add(num2);
        frame.add(operator);
        frame.add(calculate);
        frame.add(result);
```

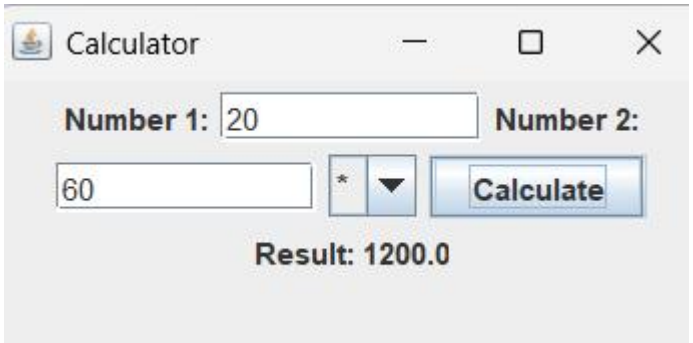
```
Welcome | ThreadSequential.java | TextFieldValidation.java | ThreadComparisonInput.java
GUICalculator.java > ...
public class GUICalculator {
    public static void main(String[] args) {
        calculate.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    double n1 = Double.parseDouble(num1.getText());
                    double n2 = Double.parseDouble(num2.getText());
                    String op = (String) operator.getSelectedItem();
                    double res = 0;
                    switch (op) {
                        case "+": res = n1 + n2; break;
                        case "-": res = n1 - n2; break;
                        case "*": res = n1 * n2; break;
                        case "/":
                            if (n2 == 0) {
                                result.setText(text: "Error: Divide by zero!");
                                return;
                            }
                            res = n1 / n2;
                            break;
                    }
                    result.setText("Result: " + res);
                } catch (Exception ex) {
                    result.setText(text: "Error: Invalid input!");
                }
            }
        });
        frame.setVisible(b: true);
    }
}
```

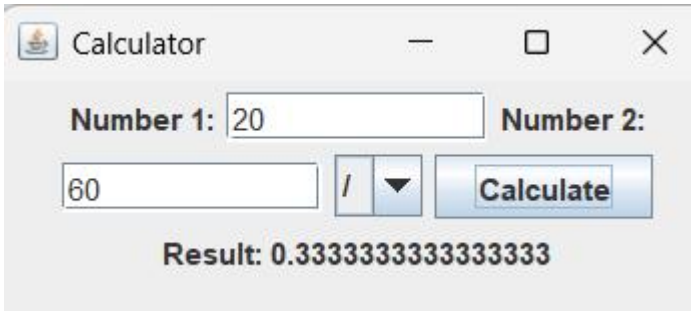
OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>javac GUICalculator.java
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>java GUICalculator.java
```









6. Demonstrate thread lifecycle states and print them.
Hint: getState().

JAVA CODE:

ThreadSequential.java ● TextFieldValidation.java ● ThreadComparisonInput.java ● FileDownloadSimulation.ja

ThreadLifecycleDemo.java > ...

```java
class MyThread extends Thread {
    public void run() {
        try {
            System.out.println("Inside run() - State: " + Thread.currentThread().getState());

            Thread.sleep(millis: 500); // TIMED_WAITING state
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}
public class ThreadLifecycleDemo {
    Run | Debug
    public static void main(String[] args) {
        MyThread t = new MyThread();
        System.out.println("After creation: " + t.getState());
        t.start();
        System.out.println("After start(): " + t.getState());
        try {
            Thread.sleep(millis: 100); // give time for thread to start
            System.out.println("During execution: " + t.getState());
            t.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
        System.out.println("After completion: " + t.getState());
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>javac ThreadLifecycleDemo.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>java ThreadLifecycleDemo.java
After creation: NEW
After start(): RUNNABLE
Inside run() - State: RUNNABLE
During execution: TIMED_WAITING
After completion: TERMINATED
```

C. HARD LEVEL

7. Simulate ticket booking with multiple threads and limited seats. Prevent overbooking.

Hint: Shared resource handling.

<u>JAVA CODE:</u>

TextFieldValidation.java ● ThreadComparisonInput.java ● FileDownloadSimulation.java ● GUICalculator.jav

TicketBookingSystem.java > TicketBookingSystem > main(String[])

```java
class TicketBooking {
    int availableSeats;
    TicketBooking(int seats) {
        availableSeats = seats;
    }
    synchronized void bookTicket(String user, int seats) {
        System.out.println(user + " trying to book " + seats + " seats...");

        if (seats <= availableSeats) {
            System.out.println("Booking successful for " + user);
            availableSeats -= seats;
            System.out.println("Seats left: " + availableSeats);
        } else {
            System.out.println("Booking failed for " + user + " (Not enough seats)");
        }
    }
}
class BookingThread extends Thread {
    TicketBooking tb;
    String user;
    int seats;
    BookingThread(TicketBooking tb, String user, int seats) {
        this.tb = tb;
        this.user = user;
        this.seats = seats;
    }
    public void run() {
        tb.bookTicket(user, seats);
    }
}
```

TextFieldValidation.java ● ThreadComparisonInput.java ● FileDownloadSimulation.java ● GU

TicketBookingSystem.java > ...

```java
public class TicketBookingSystem {
    Run | Debug
    public static void main(String[] args) {
        TicketBooking tb = new TicketBooking(seats: 5);
        BookingThread t1 = new BookingThread(tb, user: "User1", seats: 2);
        BookingThread t2 = new BookingThread(tb, user: "User2", seats: 3);
        BookingThread t3 = new BookingThread(tb, user: "User3", seats: 2);
        t1.start();
        t2.start();
        t3.start();
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>javac TicketBookingSystem.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>java TicketBookingSystem.java
User1 trying to book 2 seats...
Booking successful for User1
Seats left: 3
User2 trying to book 3 seats...
Booking successful for User2
Seats left: 0
User3 trying to book 2 seats...
Booking failed for User3 (Not enough seats)
```

8. Design GUI where JComboBox selection triggers different operations.
Hint: Event-driven design

JAVA CODE:.

ThreadComparisonInput.java ● FileDownloadSimulation.java ● GUICalculator.java ● Th

ComboBoxOperations.java > ...

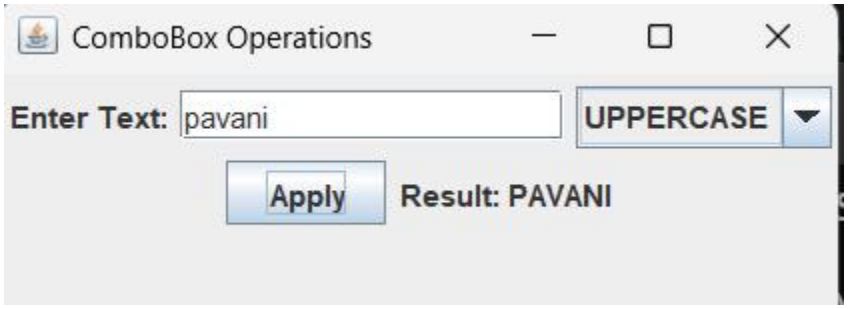
```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class ComboBoxOperations {
    Run | Debug
    public static void main(String[] args) {
        JFrame frame = new JFrame(title: "ComboBox Operations");
        frame.setSize(width: 350, height: 200);
        frame.setLayout(new FlowLayout());
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JTextField textField = new JTextField(columns: 15);
        String options[] = {"UPPERCASE", "lowercase", "Length"};
        JComboBox<String> comboBox = new JComboBox<>(options);
        JButton button = new JButton(text: "Apply");
        JLabel result = new JLabel(text: "Result: ");
        frame.add(new JLabel(text: "Enter Text:"));
        frame.add(textField);
        frame.add(comboBox);
        frame.add(button);
        frame.add(result);
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String input = textField.getText();
                String selected = (String) comboBox.getSelectedItem();
                if (input.trim().isEmpty()) {
                    result.setText(text: "Error: Enter some text!");
                    return;
                }
```

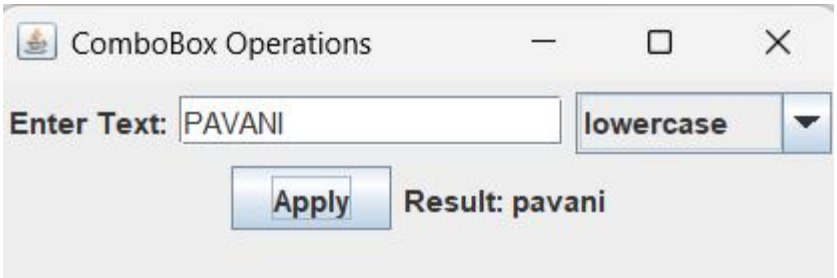
ThreadComparisonInput.java ● | FileDownloadSimulation.java ● | GUICalculator.java ● | Threa

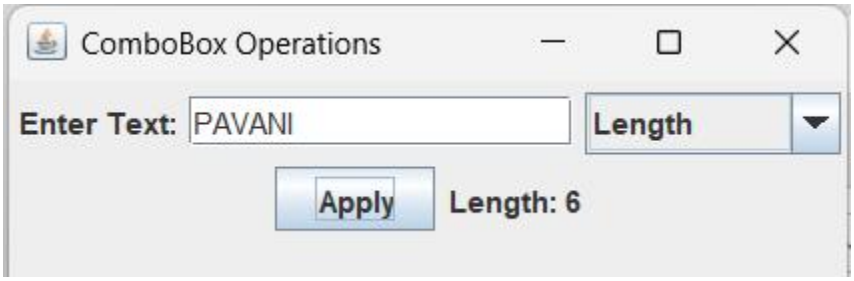
ComboBoxOperations.java > ...

```java
4   public class ComboBoxOperations {
5       public static void main(String[] args) {
20          button.addActionListener(new ActionListener() {
21              public void actionPerformed(ActionEvent e) {
28                  switch (selected) {
29                      case "UPPERCASE":
30                          result.setText("Result: " + input.toUpperCase());
31                          break;
32                      case "lowercase":
33                          result.setText("Result: " + input.toLowerCase());
34                          break;
35                      case "Length":
36                          result.setText("Length: " + input.length());
37                          break;
38                  }
39              }
40          });
41          frame.setVisible(b: true);
42      }
43  }
```

OUTPUT:




ComboBox Operations
Enter Text:
PAVANI
lowercase
Apply
Result: pavani


ComboBox Operations
Enter Text:
PAVANI
Length
Apply
Length: 6

9. Create system with three threads: compute, log, display running concurrently.
Hint: Runnable.

JAVA CODE:

FileDownloadSimulation.java ● GUICalculator.java ● ThreadLifecycleDemo.java ● TicketB

MultiThreadSystem.java > MultiThreadSystem > main(String[])

```java
class ComputeTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Compute: Square of " + i + " = " + (i * i));
            try {
                Thread.sleep(millis: 500);
            } catch (InterruptedException e) {}
        }
    }
}
class LogTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Log: Writing log entry " + i);
            try {
                Thread.sleep(millis: 700);
            } catch (InterruptedException e) {}
        }
    }
}
class DisplayTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Display: Showing data " + i);
            try {
                Thread.sleep(millis: 600);
            } catch (InterruptedException e) {}
        }
    }
}
```

FileDownloadSimulation.java ● GUICalculator.java ● ThreadLifecycleDem

MultiThreadSystem.java > ...

```java
}

public class MultiThreadSystem {
    Run | Debug
    public static void main(String[] args) {
        Runnable compute = new ComputeTask();
        Runnable log = new LogTask();
        Runnable display = new DisplayTask();
        Thread t1 = new Thread(compute);
        Thread t2 = new Thread(log);
        Thread t3 = new Thread(display);
        t1.start();
        t2.start();
        t3.start();
        try {
            t1.join();
            t2.join();
            t3.join();
        } catch (InterruptedException e) {}
        System.out.println(x: "All tasks completed!");
    }
}
```

OUTPUT:

```
C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>javac MultiThreadSystem.java

C:\Users\pavan\OneDrive\Desktop\SS\ch.ai.u4aid25028\Java_Lab_Sheet_10>java MultiThreadSystem.java
Display: Showing data 1
Log: Writing log entry 1
Compute: Square of 1 = 1
Compute: Square of 2 = 4
Display: Showing data 2
Log: Writing log entry 2
Compute: Square of 3 = 9
Display: Showing data 3
Log: Writing log entry 3
Compute: Square of 4 = 16
Display: Showing data 4
Compute: Square of 5 = 25
Log: Writing log entry 4
Display: Showing data 5
Log: Writing log entry 5
All tasks completed!
```

10. Design GUI that captures keyboard and mouse events simultaneously.
Hint: KeyListener and MouseListener.

JAVA CODE:

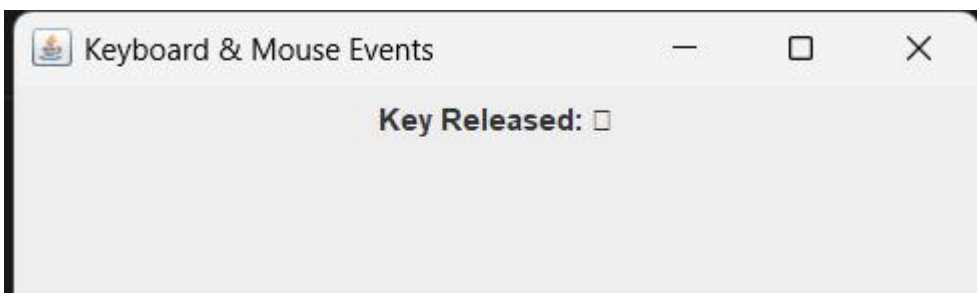
a ● J GUICalculator.java ● J ThreadLifecycleDemo.java ● J TicketBookingSystem.java ● J ComboBoxOpe

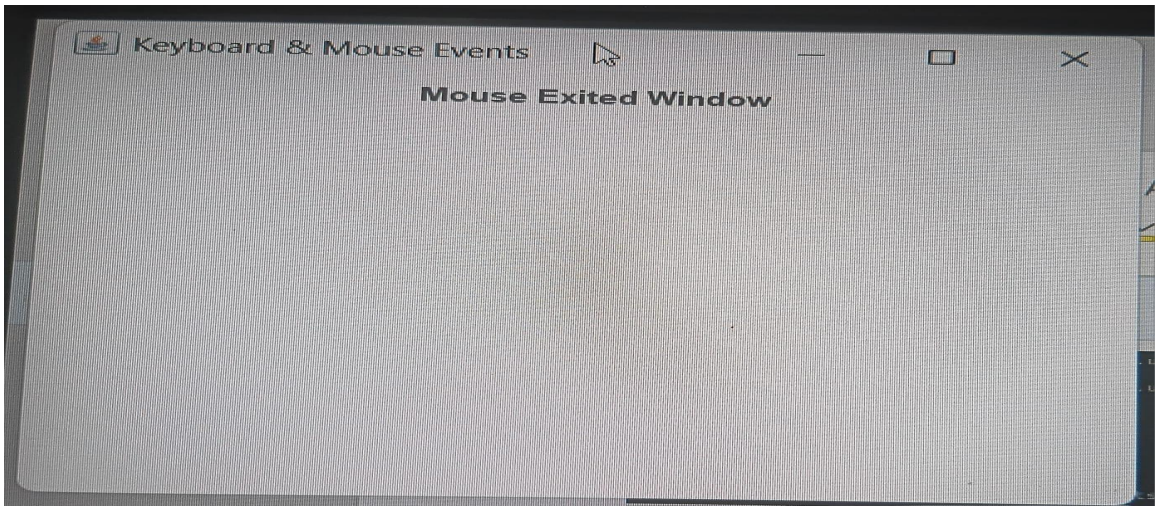
J EventCaptureGUI.java > ...

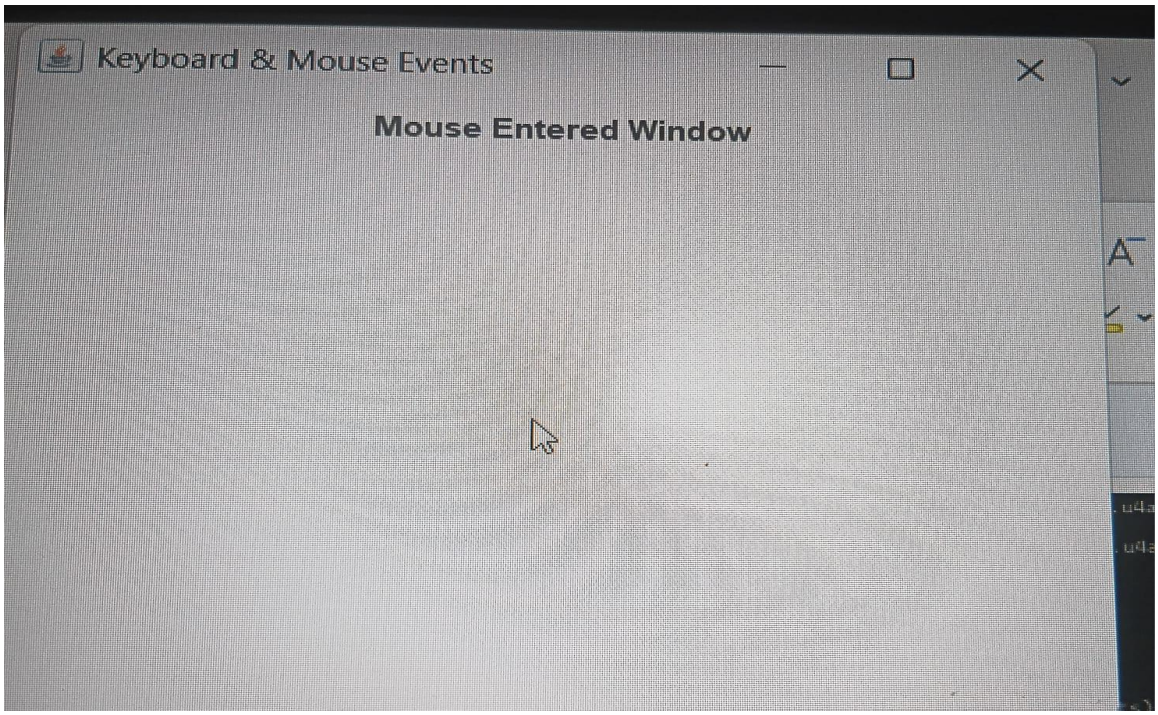
```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class EventCaptureGUI extends JFrame implements KeyListener, MouseListener {
    JLabel label;
    EventCaptureGUI() {
        setTitle(title: "Keyboard & Mouse Events");
        setSize(width: 400, height: 300);
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        label = new JLabel(text: "Perform an action...");
        add(label);
        addKeyListener(this);
        addMouseListener(this);
        setFocusable(focusable: true);
    }
    public void keyPressed(KeyEvent e) {
        label.setText("Key Pressed: " + e.getKeyChar());
    }
    public void keyReleased(KeyEvent e) {
        label.setText("Key Released: " + e.getKeyChar());
    }
    public void keyTyped(KeyEvent e) {
        label.setText("Key Typed: " + e.getKeyChar());
    }
    public void mouseClicked(MouseEvent e) {
        label.setText("Mouse Clicked at (" + e.getX() + ", " + e.getY() + ")");
    }
```

```
GUICalculator.java    ThreadLifecycleDemo.java    TicketBookingSystem.java    ComboBox
EventCaptureGUI.java > ...
public class EventCaptureGUI extends JFrame implements KeyListener, MouseListener {
    public void mousePressed(MouseEvent e) {
        label.setText(text: "Mouse Pressed");
    }
    public void mouseReleased(MouseEvent e) {
        label.setText(text: "Mouse Released");
    }
    public void mouseEntered(MouseEvent e) {
        label.setText(text: "Mouse Entered Window");
    }
    public void mouseExited(MouseEvent e) {
        label.setText(text: "Mouse Exited Window");
    }
    Run | Debug
    public static void main(String[] args) {
        new EventCaptureGUI().setVisible(b: true);
    }
}
```

OUTPUT:




Keyboard & Mouse Events
Mouse Exited Window


Keyboard & Mouse Events
Mouse Entered Window

5. ASSIGNMENT EVALUATION

0 : Not Done
1 : Incomplete
2 : Late Completion
3 : Needs Improvement
4 : Complete
5 : Well Done

6. VIVA QUESTIONS

1. What is a thread?

2. What is multithreading?

3. What is thread lifecycle?

4. What is Applet?

5. What is event handling?

6. What is delegation event model?

7. What is JButton?

8. What is JTextField?